

Day 30: Product List

Day 30 — Product List

1. Day Number

Day 30

2. Topic Name

List of Product Dictionaries

Today you will learn how to store **multiple products together** using:

- a **list**
 - multiple **dictionaries**
 - a **loop**
-

3. Connection to Yesterday

Yesterday, you created **one product** using a dictionary.

For example, conceptually:

```
Product
├── name
├── price
└── stock
```

Today, instead of storing only one product, you will store **multiple products**.

```
Yesterday:

product
  ↓
{name, price, stock}

Today:

products
  ↓
[
  {product 1},
  {product 2},
  {product 3}
]
```

So the progression is:

```
Dictionary
  ↓
One Product
  ↓
```

List of Dictionaries

↓

Multiple Products

4. Important Topics

List

A list stores multiple values together.

```
numbers = [10, 20, 30]
```

A list can also contain dictionaries.

```
items = [  
    {"name": "Pen"},  
    {"name": "Book"}  
]
```

Dictionary

A dictionary stores information as **key-value pairs**.

```
{"name": "Pen", "price": 10}
```

Here:

- "name" is a key
- "Pen" is its value
- "price" is a key
- 10 is its value

Loop

A loop lets us process every product one by one.

Conceptually:

```
Take first product  
↓  
Print information  
  
Take second product  
↓  
Print information  
  
Take third product  
↓  
Print information
```

5. Foundational Notes

Suppose you have three products.

Without a list, you might create three separate variables:

```
product1 = {...}
product2 = {...}
product3 = {...}
```

This becomes difficult when there are many products.

Instead, Python lets us put all the dictionaries inside **one list**.

```
products
├── product dictionary 1
├── product dictionary 2
└── product dictionary 3
```

A product dictionary might look conceptually like:

```
{
    "name": ...,
    "price": ...,
    "stock": ...
}
```

Then the complete data structure becomes:

```
List
├── Dictionary
│   ├── name
│   ├── price
│   └── stock
├── Dictionary
│   ├── name
│   ├── price
│   └── stock
└── Dictionary
    ├── name
    ├── price
    └── stock
```

The important idea is:

The list stores the products, and each dictionary stores the details of one product.

6. Easy Example

Imagine we want to store student information instead of products.

```
students = [  
    {"name": "Amit", "age": 20},  
    {"name": "Neha", "age": 21}  
]
```

You could process each student using a loop:

```
for student in students:  
    print(student["name"])
```

Conceptually:

```
students list  
    ↓  
First dictionary  
{ "name": "Amit", "age": 20 }  
    ↓  
student["name"]  
    ↓  
Amit  
  
Second dictionary  
{ "name": "Neha", "age": 21 }  
    ↓  
student["name"]  
    ↓  
Neha
```

You will use the same idea for products.

7. Problem Statement

Create a **list containing three product dictionaries**.

Each product should contain:

```
name  
price  
stock
```

Then use a **loop** to print each product's:

```
name  
price
```

For example, your data could represent products such as:

```
Laptop  
Mouse  
Keyboard
```

You do **not** need to print the stock for this exercise.

8. Concepts Used

You will practice:

- Python lists
- Python dictionaries
- key-value pairs
- list containing dictionaries
- for loop
- accessing dictionary values using keys
- functions
- iterating over multiple records

The main data structure is:

```
List
  ↓
Dictionary
  ↓
Product properties
```

9. Thought Process

Before writing Python code, think about the problem like this.

Step 1: What represents one product?

A dictionary.

```
Product
↓
name
price
stock
```

Step 2: What represents multiple products?

A list.

```
products = [
    product,
    product,
    product
]
```

Step 3: How do I process every product?

Use a loop.

```
for each product in products
```

Step 4: What information should I print?

From each dictionary, retrieve:

```
name  
price
```

Step 5: Since we are using a functional approach

Put the printing logic inside a function.

Conceptually:

```
products  
  ↓  
function  
  ↓  
loop through products  
  ↓  
read name and price  
  ↓  
print
```

10. Pseudocode

```
START  
  
Create a list called products  
  
Add three product dictionaries to the list  
  
Each dictionary contains:  
    name  
    price  
    stock  
  
Create a function that accepts the product list  
  
Inside the function:  
  
    FOR each product in products  
  
        get product name  
        get product price  
  
        print name and price  
  
Call the function
```

END

11. Suggested Solving Approach

Use a **functional approach**.

Try to separate:

Data

from:

Processing

Conceptually:

```
Product Data
  ↓
products list
    ↓
Function
  ↓
Loop
  ↓
Print product details
```

A possible function responsibility could be:

```
display_products(products)
```

The function should:

1. receive the list
2. loop through it
3. print each product's name and price

This is better than putting all the logic directly at the top level because later the same function can work with:

```
3 products
10 products
100 products
```

12. Easy Edge Cases

Empty Product List

Your list could contain no products.

```
products = []
```

In that situation:

```
FOR each product in products
```

has nothing to process.

Therefore, the loop simply runs **zero times**.

This is an important property of Python loops.

```
[]  
↓  
for loop  
↓  
No items  
↓  
No product printed
```

Later, you can learn to print something such as:

```
No products available
```

But for today's exercise, simply understand how an empty list behaves.

13. Expected Input

For today's exercise, you do **not** need keyboard input using `input()`.

Create the data directly in Python.

Conceptually:

```
products =  
  
Product 1:  
    name = Laptop  
    price = 50000  
    stock = 5  
  
Product 2:  
    name = Mouse  
    price = 500  
    stock = 20  
  
Product 3:  
    name = Keyboard  
    price = 1500  
    stock = 10
```

So your program's data structure should resemble:


```
[
    {product 1 information},
    {product 2 information},
    {product 3 information}
]
```

14. Expected Output

For the sample products above, your output could look like:

```
Laptop - 50000
Mouse - 500
Keyboard - 1500
```

The important part is that the program prints the **name and price of all three products using a loop**.

15. Hint Only

Think of the structure in two levels:

```
products
  ↓
LIST
  ↓
individual product
  ↓
DICTIONARY
  ↓
"name"
"price"
"stock"
```

Your loop will look conceptually like:

```
for product in products:
    # access product name
    # access product price
```

Inside the loop, remember yesterday's dictionary syntax:

```
dictionary["key"]
```

So ask yourself:

```
How do I get the "name" from product?

How do I get the "price" from product?
```

Then place that loop inside a function that receives products.

Do not create three separate print statements. The purpose of Day 30 is to let the **loop** handle all three products.